NETAJI SUBHAS UNIVERSITY OF TECHNOLOGY , DELHI

DEPARTMENT OF ELECTRONICS & COMMUNICATION

SESSION(2025-2026)

ECCOC603 Machine Learning and Artificial Intelligence

Project Documentation Report

" LeetCode Contest Cheater Detector"

Submitted By –

Shivam (2023UEC2581)
Karan Sharma (2023UEC2600)
Arnav Saxena (2023UEC2607)

I. Introduction

LeetCode is a popular online platform designed to help you learn programming, practice coding problems, and prepare for technical interviews. It's widely used by students and developers to build problem-solving skills in languages like Python, C++, Java, and more. Leetcode weekly contests are inevitable part of the platform that help users to put their programming skill at test with thousands of people across the globe in real time.

Contest Format

- **Duration:** Approximately 90 minutes
- **Number of Problems:** 4
- **Difficulty Levels:**
  - Problem 1: Easy
  - Problem 2: Easy to Medium
  - Problem 3: Medium
  - Problem 4: Hard

Scoring and Ranking

Participants are ranked based on:

- Number of problems solved
- Time taken to solve each problem
- Penalty incurred due to incorrect submissions

Faster and correct submissions result in higher rankings.

Rating System

After each contest, participants receive a rating that reflects their performance. A strong performance leads to an increase in rating, while a weaker performance may decrease it. This rating serves as an indicator of a participant's competitive coding ability.

Benefits of Participation

- Improves problem-solving speed and accuracy
- Provides real-time competitive experience
- Helps in interview preparation
- Enables performance benchmarking against global participants

A. Motivation

With the rapid advancement of large language models (LLMs) for code generation, there has been a growing concern regarding unfair practices in online coding contests. Some participants misuse these tools during contests, leading to artificially inflated rankings.

Such practices undermine the primary objective of these competitions, which is to accurately assess a participant's problem-solving ability, coding skills, and performance under time constraints. As a result, the rankings may no longer reliably reflect the true capabilities and potential of genuine participants.

This issue highlights the need for improved monitoring mechanisms and mass reporting of these users to the platform

B. Existing Solution

LeetCode employs its own anti-cheat detection mechanisms to identify unfair practices during contests. However, these systems are not fully effective and may fail to detect a significant number of violations, identifying only a limited subset of participants engaging in such behavior.

With the introduction of the code replay feature, it has become easier to observe instances where solutions appear to be copied and pasted rather than developed during the contest. Despite this increased transparency, such participants may still retain top positions on the leaderboard, raising concerns about the fairness and reliability of the ranking system.

Fig. 1: Contest 494 leaderboard as on 23 April 2026

Contest 494 was conducted on 22 March 2026. Even after a month, several participants who can be reasonably identified as potential cheaters—based on their code replay behavior and inconsistencies with their past performance—continue to occupy top positions on the leaderboard.

This persistence of suspected unfair activity further highlights the limitations of existing detection mechanisms and raises concerns about the credibility of contest rankings.

C. Goal

The objective of this project is to identify potential instances of unfair behavior by analyzing participants' profile statistics and contest performance patterns(excluding replays). Once such participants are identified, they can be flagged for further review and reported, enabling appropriate actions to be taken to maintain the integrity of the leaderboard.

II. Dataset and Features

Due to the unavailability of any relevant dataset on platforms such as Kaggle or other public repositories, the dataset for this project was constructed by extracting data from recent LeetCode contest leaderboards along with corresponding participant profile information.

A. Data Extraction

LeetCode provides publicly accessible APIs that can be utilized to retrieve contest and user-related data. Specifically, REST APIs were used to obtain contest leaderboard information, while GraphQL APIs were used to fetch detailed user profile data.

The data extraction process was implemented using Python, leveraging the `curl_cffi` module to facilitate efficient requests while ensuring compatibility with LeetCode's Cloudflare-protected endpoints.

The APIs returned data in JSON format, which was initially unstructured and required preprocessing. This data was subsequently cleaned and transformed to extract relevant features for each contest participant, forming the basis of the dataset used in this study.

B. Extracted Features

For a given Contest , the following information about the participant was collected

Feature	Description
Rank	Rank achieved in the contest
User_slug	Unique user identifier on the platform
Score	Total score obtained in the contest
CurrConSolved	Number of problems solved in the current contest
FinishTimeStamp	Time at which the participant finished the contest
Q1TimeStamp	Submission time of Question 1
Q2TimeStamp	Submission time of Question 2
Q3TimeStamp	Submission time of Question 3
Q4TimeStamp	Submission time of Question 4
Q1FailCnt	Number of failed attempts for Question 1
Q2FailCnt	Number of failed attempts for Question 2
Q3FailCnt	Number of failed attempts for Question 3
Q4FailCnt	Number of failed attempts for Question 4
UserName	Name of the participant
CountryName	Country of the participant
RankingP	Ranking based on Solved problems
PostViewCnt	Number of post views by the user
Reputation	Reputation score of the user
TotalQsAcc	Total number of accepted submissions
TotalQsSub	Total number of submitted solutions
EasyQsAcc	Accepted easy-level problems
EasyQsSub	Submitted easy-level problems
MedQsAcc	Accepted medium-level problems
MedQsSub	Submitted medium-level problems
HardQsAcc	Accepted hard-level problems
HardQsSub	Submitted hard-level problems
ContestCnt	Number of contests participated in
CurrRating	Current contest rating
GlobalRank	Global ranking of the user
topPercentage	Percentage ranking among all users
ContestHistory	Historical contest performance data

TABLE I: Extracted Features and Their Descriptions

C. Dataset Details

The dataset was constructed by extracting participant data for ranks 1 to 10,000 from three recent contests (Weekly Contest 494, Weekly Contest 495, and Weekly Contest 496). Data corresponding to users from the Chinese region could not be retrieved due to access limitations.

After removing duplicate participants appearing across multiple contests, the final dataset consisted of approximately 19,000 unique users. This resulted in a dataset of size $19{,}000 \times 31$, where each row represents a participant and each column corresponds to a feature.

The dataset was then divided into training and testing subsets. The testing set comprised approximately 180 users, uniformly sampled across the three contests, while the remaining data was used for training.

The relatively small size of the testing set is attributed to the manual effort required for labeling. Identifying and verifying instances of unfair behavior is a time-consuming process, and significant effort was invested to ensure high-quality ground truth labels for the selected test samples.

It was ensured to avoid any form of data leakage in the dataset

III. EDA

The exploratory data analysis was conducted on the cleaned dataset to understand feature distributions, identify anomalies, and motivate feature engineering choices before modelling.

Score and Problems Solved

The score distribution revealed a strong right skew, with most users solving two or three problems. The `CurrConSolved` distribution followed a similar pattern, confirming that users solving all four problems form a minority. Perfect scorers were later cross-referenced with contest time and failure counts to identify suspicious behaviour.

Time Taken

The `time_taken` feature showed a heavy right tail spanning the contest duration. A focused inspection of users finishing within 1000 seconds revealed a small group completing all problems in under 500 seconds, which is highly unusual for legitimate participants and therefore useful as a fraud signal.

Experience and Rating

The `ContestCnt` distribution was heavily concentrated around first-time participants (`ContestCnt`=1). The `CurrRating` distribution ranged from below 1500 to above 3000 and resembled the typical competitive programming rating spread.

Correlation Analysis

A lower-triangular correlation heatmap was generated using 13 numerical features. A very strong positive correlation of 0.96 was observed between `Score` and `CurrConSolved`. Strong correlations (> 0.85) were also found among `Total_solved`, `Easy`, and `Medium`, indicating redundancy. These highly correlated features were reviewed, and selected redundant columns were removed from final modelling to reduce multicollinearity and improve anomaly detection sensitivity.

Engineered Feature Distributions

Four derived features were analysed:

- **rating_jump**: Most users showed a moderate positive increase. A spike at zero suggested new or potentially inactive accounts.

$$\texttt{rating_jump} = \texttt{CurrRating} - \texttt{rating_first}$$

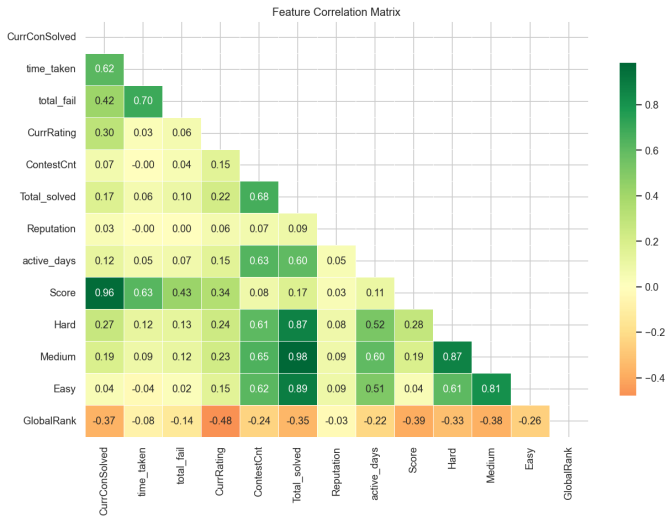

Fig. 2: Correlation Matrix

- **score_per_time**: Strongly right-skewed, with the upper tail representing extreme solving efficiency.

$$\texttt{score_per_time} = \frac{\texttt{CurrConSolved}}{\max(\texttt{time_taken}, 1)}$$

- **hard_ratio**: Values ranged from 0 to 1, indicating the proportion of hard problems solved.

$$\texttt{hard_ratio} = \frac{\texttt{Hard}}{\texttt{Total_solved} + 1}$$

- **solved_per_contest**: High values suggested long-term consistent practice.

$$\texttt{solved_per_contest} = \frac{\texttt{Total_solved}}{\texttt{ContestCnt} + 1}$$

Clean vs Suspicious Separation

Distribution overlays between clean and flagged users showed clear separation on `time_taken`, `ContestCnt`, `rating_jump`, and `score_per_time`, validating these as effective detection signals.

IV. Model Methodology

The detection pipeline is fully unsupervised and combines two components: a rule-based scoring engine and a rating-bucket Local Outlier Factor (LOF) model. Their outputs are fused into one final suspicion score.

Why Not a Standard Classifier?

Isolation Forest was tested initially as a baseline. However, it frequently marked high-rated legitimate users as anomalies because elite performance is naturally rare. This showed that global anomaly detectors cannot distinguish exceptional skill from cheating, motivating the hybrid approach.

Rule-Based Scoring Engine

Seven expert-designed rules were used. Each suspicious condition adds a weighted score:

Rule	Weight
Near-zero active days with high score	3
New account with top-200 rank	3
Zero rating jump with high rating	3
Perfect solve with few contests	2
Weak profile but top-200 rank	2
Low solved count but high rating	2
Ghost profile with high rank	2

Fig. 3: Rule-Based Suspicion Score Distribution

Rating-Stratified LOF

Users were divided into rating groups:

D ($<$ 1600), C (1600–1800), B (1800–2000), A (2000–2200), and S (2200+).

A separate `LocalOutlierFactor` model was trained for each group using:

$\{$`CurrConSolved`, `time_taken`, `total_fail`, `score_per_time`, `hard_ratio`$\}$

Neighbour count was:

$$k = \min(20, n - 1)$$

Features were standardised before training.

The raw LOF score was clipped at the global 99th percentile and scaled to $[0, 3]$. Refer to Fig 3

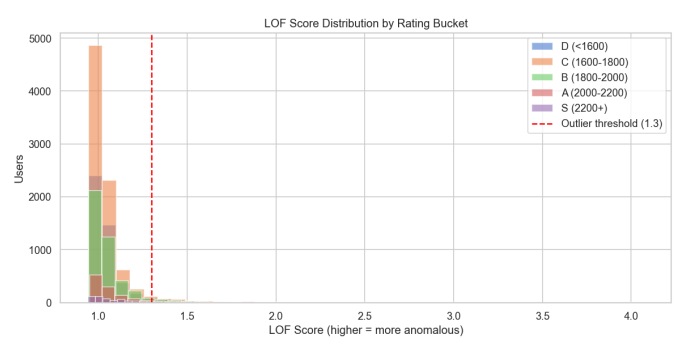

Fig. 4: LOF Score Distribution by Rating Bucket

Score Fusion and Thresholding

Final score:

$$S_{\text{final}} = S_r + 0.3 \times \ell_{\text{norm}}$$

The LOF signal was down-weighted so that rare but fair elite performances were not overly penalised.

Classification thresholds:

- $S_{\text{final}} \geq 6$: **HIGHLY SUSPICIOUS**
- $S_{\text{final}} \geq \tau$: **SUSPICIOUS**
- Otherwise : **CLEAN**

Here, τ is the 80th percentile of training scores.

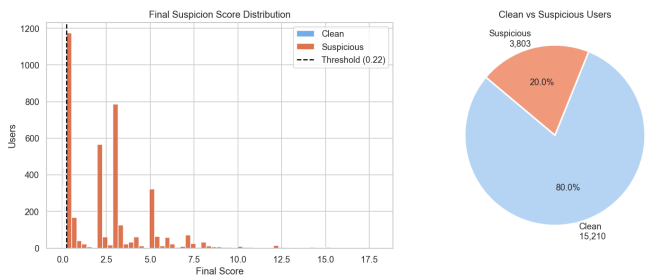

Fig. 5: Final Score Ditribution

V. Model Evaluation

The model was tested on a dataset of 180 users whose true labels were manually identified as cheater (1) or non-cheater (0). Users classified as *SUSPICIOUS* and *HIGHLY SUSPICIOUS* were considered as cheaters.

A. Confusion Matrix

	Predicted 0	**Predicted 1**
Actual 0	93 (TN)	7 (FP)
Actual 1	53 (FN)	27 (TP)

Confusion Matrix

B. Evaluation Metrics

- Accuracy = $\frac{TP+TN}{TP+TN+FP+FN} = \frac{27+93}{180} = 0.6667$
- Precision = $\frac{TP}{TP+FP} = \frac{27}{27+7} = 0.7941$
- Recall (TPR) = $\frac{TP}{TP+FN} = \frac{27}{27+53} = 0.3375$
- F1-Score = 0.4737
- False Positive Rate (FPR) = $\frac{FP}{FP+TN} = \frac{7}{7+93} = 0.07$
- False Negative Rate (FNR) = $\frac{FN}{FN+TP} = \frac{53}{53+27} = 0.6625$
- Specificity (TNR) = $\frac{TN}{TN+FP} = \frac{93}{100} = 0.93$
- Balanced Accuracy = $\frac{TPR+TNR}{2} = \frac{0.3375+0.93}{2} \approx 0.6338$

C. Comments

The model achieves an overall accuracy of 66.67%, indicating moderate predictive performance. The precision of 0.79 suggests that when the model predicts a participant as a cheater, it is correct most of the time. However, the recall is significantly low at 0.34, indicating that the model fails to identify a large proportion of actual cheaters.

The false negative rate (66.25%) is particularly high, meaning many cheaters are misclassified as genuine participants. On the other hand, the false positive rate is low (7%), which implies that genuine users are rarely misclassified as cheaters.

This indicates that the model is conservative in predicting cheating behavior—it prioritizes avoiding false accusations but at the cost of missing many actual cheaters. While this may be desirable in scenarios where false positives are critical, it limits the model's effectiveness for comprehensive detection.

Overall, the model demonstrates high precision but low recall, suggesting the need for improvements such as better feature engineering, threshold tuning, or incorporating additional behavioral signals to enhance detection capability.

VI. Conclusion

A basic interface program was developed to enable the application of the model on new contest data.

Although the model remains conservative in classifying participants as cheaters, it demonstrates improved sensitivity compared to LeetCode's existing anti-cheat mechanisms, which appear to be even more reluctant in flagging suspicious behavior.

A. Future Work

- Explore and evaluate alternative modeling approaches to improve detection performance
- Expand the size of the testing dataset to enable more robust evaluation
- Deploy the model as a user-accessible application for broader usability

VII. Contributions

Karan proposed the project idea and served as the project lead. He was responsible for data extraction from the LeetCode platform and for preparing the training and testing datasets.

Shivam handled the development of the machine learning model, including exploratory data analysis (EDA), model training, and evaluation.

Arnav developed the interface for the model and was responsible for preparing the project report.

Acknowledgment

We would like to thank Ms. Vandana Bhatia for providing us with the opportunity to explore the practical aspects of Machine Learning through this project.

We would also like to express our gratitude to LeetCode for providing free public APIs that were essential for generating the dataset.

Finally, we would like to thank our classmates who explained their projects in detail and showed interest in our work by asking about its progress.

REFERENCES

[1] Stanford University, "CS229 Final Projects (Spring 2021)," [Online]. Available: https://cs229.stanford.edu/proj2021spr/. [Accessed: Apr. 23, 2026].

[2] ScraperAPI, "Web Scraping Guide," [Online]. Available: https://www.scraperapi.com/web-scraping/. [Accessed: Apr. 23, 2026].

[3] LeetCode, "Is there public API endpoints available?," [Online]. Available: https://leetcode.com/discuss/post/1297705/is-there-public-api-endpoints-available-h0661/. [Accessed: Apr. 23, 2026].

[4] LeetCode, "Contest Platform," [Online]. Available: https://leetcode.com/contest/. [Accessed: Apr. 23, 2026].

[5] JSON Viewer, "Online JSON Viewer Tool," [Online]. Available: https://jsonviewer.stack.hu/. [Accessed: Apr. 23, 2026].

[6] scikit-learn, "Unsupervised Learning," [Online]. Available: https://scikit-learn.org/stable/unsupervised_learning.html. [Accessed: Apr. 23, 2026].